

1 Introduction

In this lab, we will implement the Viterbi decoder on a FPGA. The Viterbi decoder is used to interpret data that has been encoded with convolutional coding. We will introduce a few concepts of Viterbi encoding, and proceed to Viterbi decoding.

Convolutional coding: Given an input bitstream, we can encode it using convolutional coding. This operation involves performing a convolution on a window of length K . The output from each time convolution is carried out is the parity bit. In each clock cycle, we can do r convolutions to produce r parity bits.

For example, assume the parity bits are calculated by: (This is one example, not necessarily always true)

$$p0[n] = x[n] \oplus x[n - 1] \oplus x[n - 2]$$

$$p1[n] = x[n] \oplus x[n - 1]$$

In the above case, $r=2$ since there are two parity bits in each cycle, and $K=3$ since the largest length of the convolution window is 3 (from $x[n]$ to $x[n - 2]$). Note that the “XOR” here is the **mod 2 addition**.

Suppose a bitstream is 101110, where **the leftmost bit is the earliest bit** ($x[0]$). We assume $x[-1]$ and $x[-2]$ are 0, then the first two parity bits ($p0[0]$ and $p1[0]$) are 1 each, which gives 11. The second is 11, the third is 01, all the way to the sixth, which is 01. Therefore, the length of the output encoding code is $r \times \text{length}(\text{input bitstream})$.

Viterbi decoder: Given an encoded bitstream, the Viterbi decoder is employed to decode it. First, we need to find a reliable way to represent our encoding system. For the encoding example detailed above, we can represent it by the circuit shown in Figure 1.

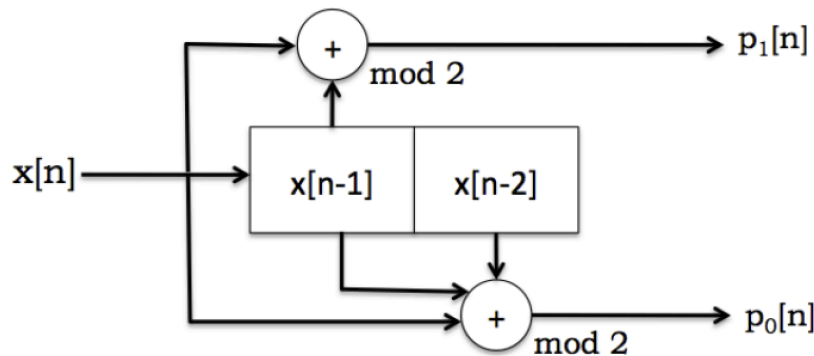


Figure 1: Viterbi Encoding System

There are two registers $x[n - 1]$ and $x[n - 2]$. Besides, we can also represent this using a finite state machine as shown in Figure 2. In Figure 2, the number in each state represents the value stored in the registers ($x[n - 1]$ and $x[n - 2]$), **with the leftmost bit representing the most recent bit** ($x[n - 1]$).

ECEN 428/722 – Lab 6: Viterbi Decoder on FPGAs

The length of the bits in each state is $K-1$. Each x/yy on edge represents that if the next bit is x , the parity bits output is yy . The length of the bits of yy is r .

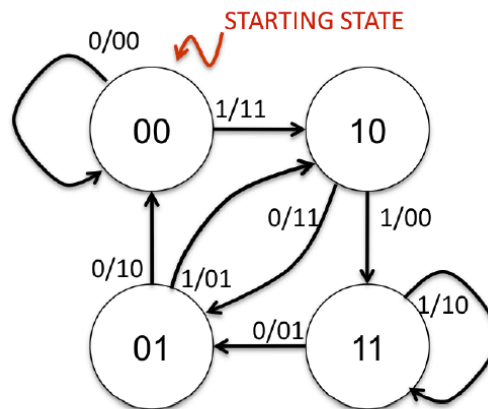


Figure 2: Finite State Machine

There exists a structure called Trellis. This structure describes the state change each time a bit is given as input. An example is shown in Figure 3. The output bitstream (yy) here is 111101000110.

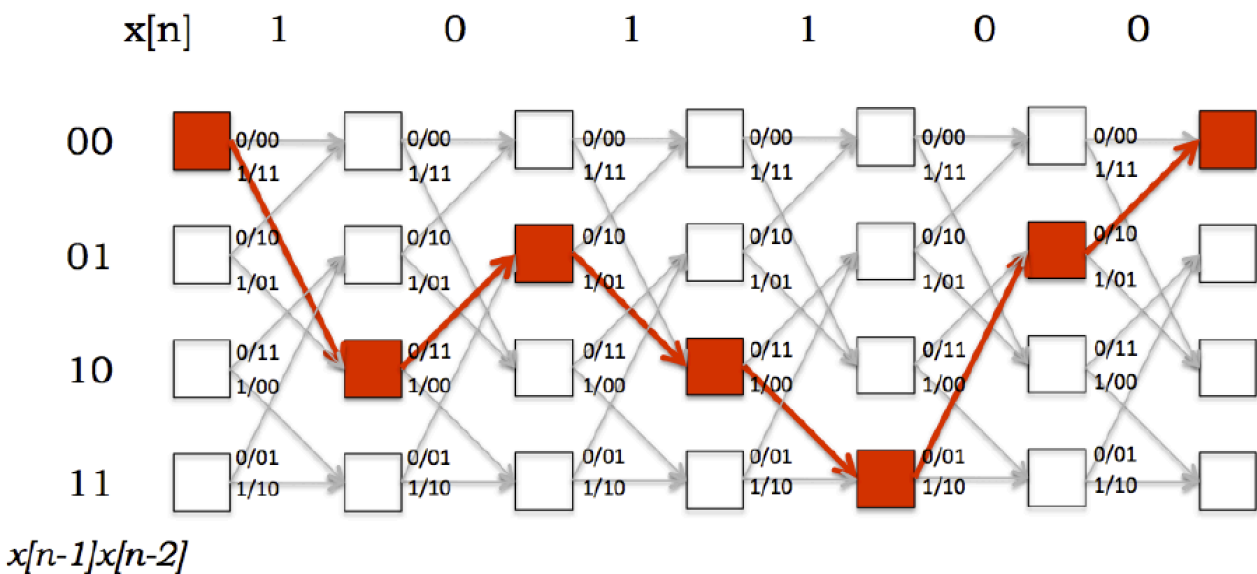


Figure 3: Trellis Structure

To decode, given an encoded bitstream, we need to find out the path in Trellis, which has the highest probability (lowest error) to generate the encoded bitstream.

There are 3 steps to decoding the Trellis structure:

- The first step is, for each of the r encoded bits, the error is calculated between itself and the output of each edge present in one column of the Trellis structure. This error is the Hamming distance. An example is shown in Figure 4.

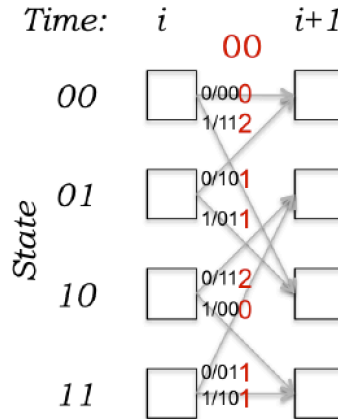


Figure 4: Hamming Distance Calculation

In the example from Figure 4, the two encoded bits (parity bits) are 00. There are two edges that can perfectly fit it with 0 error, which are edge 00 → 00, and 10 → 11. In Figure 3, you need to calculate the error 48 times in total (6 pairs of encoded bits-yy times 8 differences in each column). If the encoded stream is 111101000110, you need to calculate the error using 11 with all possible outputs in the first column and calculate the error using 11 with all possible outputs in the second column, ... until the last pair 10 with all possible outputs of the sixth column.

- The second step is to calculate the least error on each node in Trellis. Since we assume initially, all the registers ($x[n - 1] \dots$) are 0, we set the error of the state (node) 00 in the first column to 0 and all the other nodes in the first column as ∞ . Besides, all the errors of other nodes in Trellis are set as ∞ . Next, starting from column 2, we check each edge between column $x-1$ and column x . If there is an edge e from a node m in column $x-1$ and a node n in column x , and the

$$\text{error of } m + \text{error of } e < \text{error of } n$$
 then we update the *error of n* to the *error of m + error of e* and record the best edge e for node n .
- The third step is, starting from the last column, always find the node with the least error and backtrack to the first column. When backtracking one edge, the input of the edge is recorded (for instance, if an edge is “1/10”, the input is 1), and the decoded bitstream is in the reversed order of the recorded edge inputs.

In this lab, you are required to implement the Viterbi decoder with $r=2$ and $K=3$.

2 Lab Design on Viterbi Decoder

In this section, we need to implement the Viterbi module in Vivado. Before you proceed, please take the following preparation steps:

- Download “**Lab6_student_code.zip**” from the Shared Drive and **extract it**. After extraction, you will get a folder named “**Lab6_student_code/**”.
- Download “**lab6_base.zip**” from the Shared Drive and **extract it**.
- Open the project by double-clicking on “**lab6_base/base/base.xpr**”.
- Add all the source files to the project from “**Lab6_student_code/**” and implement your design according to Section 1.

Design your Viterbi decoder inside “**viterbi.v**”. You are required to design a Viterbi decoder with $r=2$ and $K=3$. The input code length is 10. Thus, the output code length is 5.

In “**viterbi.v**”, *codein* is the input code that needs to be decoded. The highest r bits (bit 9 to bit 8) are the earliest bits ($p0[0]$, $p1[0]$). The *state_out* input contains information about the state output given the input. The structure of it is: [the output when inputting 1 at state 11][the output when inputting 0 at state 11] [the output when inputting 1 at state 10][the output when inputting 0 at state 10] [the output when inputting 1 at state 01][the output when inputting 0 at state 01][the output when inputting 1 at state 00][the output when inputting 0 at state 00], with each “[]” denoting r bits. The highest bit in the state representation here is the latest bit ($x[n - 1]$). For example, if the current state is 10, the input is 1, and the next state is 11 but not 01.

You are required to write Verilog code such that:

- When *rst* is 0, *finish* is set to 0.
- When *rst* is 1, the module starts calculating.
- The decoded bitstream is finally put at the output port *codeout*. The first data is the earliest bit ($x[0]$).
- When the *codeout* is ready, *finish* is set to 1, and both *codeout* and *finish* are required to hold their values.

After you designed the decoder, run the simulation based on “**viterbi_tb.v**”. You should see the decoded bitstream.

3 Auxiliary Testbenches

In this lab, we have provided two testbenches “**bmu_tb.v**” and “**pmu_tb.v**” to help you test if your **bmu.v** module and **pmu.v** module are correctly implemented.

It is recommended to pass the testbenches before you proceed to the next section. In order to test more input cases, you can also create your own testbench based on the ones that we provided.

4 Implementation of the design on the FPGA

In this section, we will implement the design on the FPGA.

- a) Right click “**top_viterbi.v**” in the “source” panel and click “set as top” (If this file is already in bold font, it is already the top module).
- b) Now, “**top_viterbi.v**” needs a dual-port RAM. We add the RAM for “**top_viterbi.v**”.
- c) On the left panel, click *IP catalog*. On the top right corner, search “ram”. Double click “block memory generator”.
- d) If there is a window popped up asking if you want to add IP to block design or customize IP, choose customize IP.
- e) For memory type, select “**True Dual Port Ram**”.
- f) In both port A and port B options, change write width to **8**, and write depth to 65536.
- g) Operation mode “**Read First**”, enable port type “**Always Enabled**”. Click “ok”.
- h) Select “Global” in the synthesis option and click “generate” in the pop-up window.
- i) Now, click generate bitstream. After the bitstream is generated, click *file->export->export hardware*. Then, check “include bitstream”, click “ok”.
- j) Report hardware utilization, power consumption, and critical path delay as in the previous lab.
- k) Please launch SDK and generate the boot image (BOOT.bin) as in the previous lab using:
Use the bitstream file lab6_base/zybo_linux/zybo20/hw/top_viterbi.bit.
- l) Copy the updated **BOOT.bin** and **lab6_viterbi_test** into your SD card, boot the FPGA and run the test with the following command:

./lab6_viterbi_test

- m) Take a screenshot of the terminal when the result shows.
- n) Unmount the SD card, exit the serial communication and turn off your FPGA.

Some commonly used commands:

- **mount /dev/mmcbk0p1 /mnt/**
- **cd /mnt/**
- **insmod transfpga.ko**
- **mknod /dev/transfpga c 245 0**
- **./lab6_viterbi_test**
- **cd /**
- **umount /mnt/**

5 Task for Students

The following are the tasks that should be done by students.

- a) Implement the Viterbi decoder design, share the project files, the screenshot of the terminal output after running **lab6_viterbi_test**
- b) Report hardware utilization, power consumption, timing, and the throughput of your design.
- c) Ensure that the timestamp and your NetID are included in all the tool-generated log files.
- d) Answer the question:

Is Viterbi Decoder guaranteed to decode the original data correctly? Why?

6 Post-lab Submission

1. Submit a PDF file, containing the following items:
 - Screenshot of behavioral simulation of the “**viterbi_tb.v**” testbench
 - Screenshots of the terminal output for the Viterbi decoder design
 - Screenshots of the hardware utilization, power consumption, timing and throughput of your design
 - Answer to the question in Section 5.
2. Submit your “**viterbi.v**” and other relevant verilog files in a ZIP file.
3. Name the PDF file as “Lab#_PostLab_Section#_LastName_FirstName.pdf”.
4. Name the ZIP file as “Lab#_PostLab_Section#_LastName_FirstName.zip”.
5. Please submit both PDF and ZIP files on Canvas.